

PRACTICAL-4

Implementation of word embedding techniques (Bag of Words and TF-IDF) in NLP using Python

1. Bag of Words (BoW)

The Bag of Words model is one of the simplest and most widely used methods for converting text into numerical features for machine learning and NLP tasks.

The central idea is:

- Treat every sentence or document as a “bag” of words.
- Ignore grammar, sentence order, and structure.
- Focus only on whether a word occurs in a sentence (binary presence) or how many times it occurs (frequency).

How it Works in the Code:

1. Preprocessing the Text

- The input paragraph is split into sentences (`sent_tokenize`).
- Each sentence is cleaned: converted to lowercase, punctuation removed, and extra spaces normalized.

2. Tokenization and Word Count

- Sentences are split into words (`word_tokenize`).
- A dictionary (`word2count`) is built that stores the frequency of each word.
- Stopwords such as *the*, *is*, and *and* are removed since they do not add much meaning.

3. Frequency Distribution and Visualization

- Word frequencies are arranged in a pandas DataFrame.
- The top 10 frequent words are shown using a bar graph.

4. Vector Representation (BoW Matrix)

- For the most frequent words, a binary matrix is created:
 - Rows = Sentences
 - Columns = Words
 - Cell value = 1 if the word is present in the sentence, else 0.
- This is displayed as a heatmap.

Why BoW is Useful:

- Simple, fast, and easy to implement.
- Provides a baseline representation for text classification and clustering.

Limitations:

- Ignores word order → “dog bites man” and “man bites dog” look identical.
- Produces very high-dimensional, sparse matrices for large vocabularies.
- All words are treated equally without considering their importance.

CODE:-

1. Importing Libraries

```
#BAG OF WORDS
import nltk
import re
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import CountVectorizer

# Download resources
nltk.download('punkt_tab')
nltk.download('stopwords')
```

Output:

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

2. Converting paragraph to sentences.

```
text = """Beans. I was trying to explain to somebody as we were flying in, that's corn. That's beans

dataset = nltk.sent_tokenize(text)

for i in range(len(dataset)):
    dataset[i] = dataset[i].lower()
    dataset[i] = re.sub(r'\W', ' ', dataset[i])
    dataset[i] = re.sub(r'\s+', ' ', dataset[i])

for i, sentence in enumerate(dataset):
    print(f"Sentence {i+1}: {sentence}")
```

```
Sentence 1: beans
Sentence 2: i was trying to explain to somebody as we were flying in that s corn
Sentence 3: that s beans
Sentence 4: and they were very impressed at my agricultural knowledge
Sentence 5: please give it up for amaury once again for that outstanding introduction
Sentence 6: i have a bunch of good friends here today including somebody who i served
Sentence 7: i also noticed by the way former governor edgar here who i haven t seen in
Sentence 8: and it s great to see you governor
Sentence 9: i want to thank president killeen and everybody at the u of i system for m
Sentence 10: and i am deeply honored at the paul douglas award that is being given to
Sentence 11: he is somebody who set the path for so much outstanding public service he
Sentence 12: now i want to start by addressing the elephant in the room
Sentence 13: i know people are still wondering why i didn t speak at the commencement
```

3. Stop word removal and measuring frequencies

```
import pandas as pd

word2count = {}

for data in dataset:
    words = nltk.word_tokenize(data)
    for word in words:
        if word not in word2count:
            word2count[word] = 1
        else:
            word2count[word] += 1

stop_words = set(stopwords.words('english'))

filtered_word2count = {word: count for word, count in word2count.items() if word not in stop_words}

word_freq_df = pd.DataFrame(list(filtered_word2count.items()), columns=['Word', 'Frequency'])

word_freq_df = word_freq_df.sort_values(by='Frequency', ascending=False)

print(word_freq_df)
```

Output:

	Word	Frequency
3	somebody	3
0	beans	2
17	today	2
12	outstanding	2
41	want	2
..	...	...
67	people	1
68	still	1
69	wondering	1
70	speak	1
71	commencement	1

[72 rows x 2 columns]

4. Plotting the most frequent words.

```
import heapq
import matplotlib.pyplot as plt

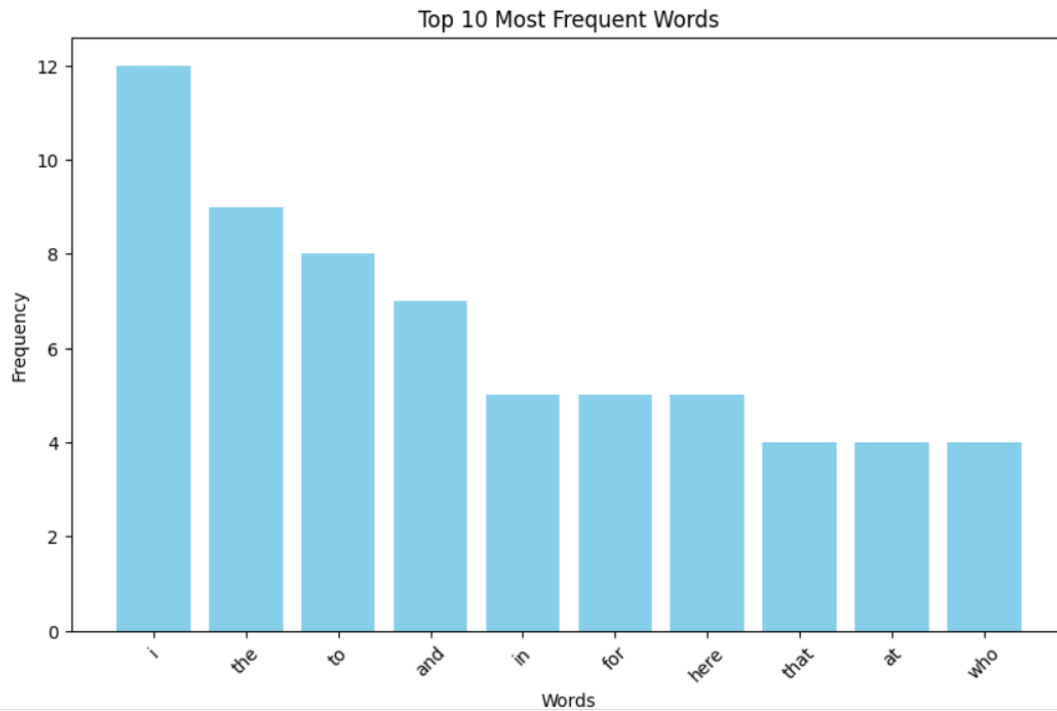
freq_words = heapq.nlargest(10, word2count, key=word2count.get)

print(f"Top 10 frequent words: {freq_words}")

top_words = sorted(word2count.items(), key=lambda x: x[1], reverse=True)[:10]
words, counts = zip(*top_words)

plt.figure(figsize=(10, 6))
plt.bar(words, counts, color='skyblue')
plt.xticks(rotation=45)
plt.title('Top 10 Most Frequent Words')
plt.xlabel('Words')
plt.ylabel('Frequency')
plt.show()
```

Top 10 frequent words: ['i', 'the', 'to', 'and', 'in', 'for', 'here', 'that', 'at', 'who']



5. Bag of word Matrix

```
import numpy as np
import seaborn as sns

X = []

for data in dataset:
    vector = []
    for word in freq_words:
        if word in nltk.word_tokenize(data):
            vector.append(1)
        else:
            vector.append(0)
    X.append(vector)

X = np.asarray(X)

plt.figure(figsize=(10, 6))
sns.heatmap(X, cmap='RdYlGn', cbar=False, annot=True, fmt="d", xticklabels=freq_words, yticklabels=[
    ])

plt.title('Bag of Words Matrix')
plt.xlabel('Frequent Words')
plt.ylabel('Sentences')
plt.show()
```

Bag of Words Matrix

Sentence 1	0	0	0	0	0	0	0	0	0	0
Sentence 2	1	0	1	0	1	0	0	1	0	0
Sentence 3	0	0	0	0	0	0	0	1	0	0
Sentence 4	0	0	0	1	0	0	0	0	1	0
Sentence 5	0	0	0	0	0	1	0	1	0	0
Sentence 6	1	1	1	1	1	0	1	0	0	1
Sentence 7	1	1	0	1	1	0	1	0	0	1
Sentence 8	0	0	1	1	0	0	0	0	0	0
Sentence 9	1	1	1	1	0	1	1	0	1	0
Sentence 10	1	1	1	1	0	0	0	1	1	0
Sentence 11	0	1	0	0	1	1	1	0	0	1
Sentence 12	1	1	1	0	1	0	0	0	0	0
Sentence 13	1	1	0	0	0	0	0	0	1	0
	i	the	to	and	in	for	here	that	at	who

Frequent Words

2. Term Frequency – Inverse Document Frequency (TF-IDF)

The TF-IDF model builds on BoW but adds a measure of importance for words. Instead of just counting, it evaluates how significant a word is to a particular document within a collection.

Key Concepts:

1. Term Frequency (TF):

Measures how frequently a word appears in a document.

$$TF(t, d) = \frac{\text{Count of term } t \text{ in document } d}{\text{Total terms in document } d}$$

2. Inverse Document Frequency (IDF):

Reduces the weight of very common words that appear across many documents.

$$IDF(t) = \log \frac{\text{Total number of documents}}{1 + \text{Number of documents containing } t}$$

3. TF-IDF Score:

Combines the two:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

This ensures:

- Words frequent in one document but rare across others get high weight.
- Common words (like “the”, “is”) get low weight.

Code:-

```
#TF-IDF
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [ "The quick brown fox jumps over the lazy dog.",
              "A journey of a thousand miles begins with a single step." ]

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)
feature_names = vectorizer.get_feature_names_out()
tfidf_values = {}

for doc_index, doc in enumerate(documents):
    feature_index = tfidf_matrix[doc_index, :].nonzero()[1]
    tfidf_doc_values = zip(feature_index, [tfidf_matrix[doc_index, x] for x in feature_index])
    tfidf_values[doc_index] = {feature_names[i]: value for i, value in tfidf_doc_values}

for doc_index, values in tfidf_values.items():
    print(f"Document {doc_index + 1}:")
    for word, tfidf_value in values.items():
        print(f"{word}: {tfidf_value}")
    print("\n")
```

Output

```
Document 1:
the: 0.6030226891555273
quick: 0.30151134457776363
brown: 0.30151134457776363
fox: 0.30151134457776363
jumps: 0.30151134457776363
over: 0.30151134457776363
lazy: 0.30151134457776363
dog: 0.30151134457776363

Document 2:
journey: 0.3535533905932738
of: 0.3535533905932738
thousand: 0.3535533905932738
miles: 0.3535533905932738
begins: 0.3535533905932738
with: 0.3535533905932738
single: 0.3535533905932738
step: 0.3535533905932738
```